PRODUCT REQUIREMENTS DOCUMENT

SonosWidget

A Jetpack Glance Widget for Pixel 9

Full-Featured Sonos Control Without Leaving Your Home Screen

Field	Value
Author	Owen (Sycamore Creek Consulting)
Version	2.0
Date	March 15, 2026
Status	**Final — Agent-Ready Build Spec (Phase 1 Complete)**
Platform	Android 16 (API 36) / Pixel 9
Type	Personal Project • Kotlin • Jetpack Glance • Sonos API

Table of Contents

1. Executive Summary
2. Problem Statement
3. Goals & Success Metrics
4. User Personas & Scenarios
5. Feature Requirements
6. Widget Layout & Sizing
7. Interaction Design
8. Visual Design System
9. Technical Architecture
10. API & Authentication Strategy
11. Data Flow & State Management
12. Performance & Battery Budget
13. Error Handling & Edge Cases
14. Accessibility
15. Security Considerations
16. Agent Build Plan & Task Decomposition
17. Tech Stack & Dependency Manifest
18. Open Questions & Risks
19. Appendix: Engineering Review Decision Log
20. Appendix: Agent Workflow Guidelines

1. Executive Summary

SonosWidget is a native Android home screen widget built with Jetpack Glance that provides comprehensive Sonos system control directly from the Pixel 9 home screen. The widget dynamically scales between three configurations — a hyper-compact mini layout for the lock screen, a half-screen (4×2) layout, and a full-screen (4×5) layout — offering real-time now-playing information with album art, playback controls, volume adjustment, speaker grouping, queue management, and music source switching — all without launching the Sonos app.

The widget communicates with Sonos hardware using a local-first architecture: SSDP and mDNS discovery race concurrently (2-second cap), with tiered fallback to manually configured IPs and then the Sonos Cloud API (OAuth 2.0). Direct UPnP/SOAP control over the local network provides minimal latency. A lightweight companion app handles authentication, preferences, and provides a foreground service for real-time playback state updates.

This is a personal-use project targeting a single Pixel 9 running Android 16 (API 36), written entirely in Kotlin. The project compiles against API 36 to leverage Android 16-specific capabilities: Local Network Protections (new runtime permission for LAN access required by UPnP/mDNS discovery) and Glance 1.2 (image alpha support for badge overlays). Widget picker previews use `android:previewLayout` (standard Android 12+ XML mechanism) to display a representative now-playing state. Rich haptic feedback uses VibrationEffect.Composition with haptic primitives (API 31+) for distinct tactile profiles on playback controls and volume adjustment, with simple one-shot vibration fallback on older devices.

Implementation is executed by coding agents (e.g., Claude Code), not human developers directly. The PRD is structured accordingly: Section 16 decomposes the build into discrete, context-window-friendly agent tasks with explicit acceptance criteria, file inventories, and verification gates between each step. Section 20 provides agent workflow guidelines.

2. Problem Statement

Controlling a Sonos system from an Android device currently requires launching the Sonos app, which involves a full context switch: unlocking, finding the app, waiting for it to load and discover speakers, then navigating to the desired control. For frequent actions like pausing music when the doorbell rings, adjusting volume during a call, or grouping speakers for a gathering, this multi-step process creates unnecessary friction.

Android's media notification provides basic play/pause, but lacks volume control, speaker grouping, queue visibility, and source switching. There is no widget-level solution that provides meaningful Sonos control without leaving the home screen.

2.1 Core Insight

The most common Sonos interactions — pause, volume, skip, group — are quick, intent-driven actions. They should live at the OS surface level (home screen), not behind an app launch. A well-designed widget can handle 80% or more of daily Sonos interactions without a single app open.

3. Goals & Success Metrics

3.1 Primary Goals

1. Eliminate app launches for routine Sonos control (play/pause, volume, skip, group).
2. Provide a glanceable, beautiful now-playing display with adaptive album art theming.
3. Support full speaker grouping and ungrouping for a 3–5 room Sonos system.
4. Deliver near-instant feedback for all control actions: < 100ms haptic response, < 500ms visual state update.
5. Maintain minimal battery impact through intelligent update strategies.

3.2 Success Metrics

Metric	Target	Measurement
Daily app launches avoided	80%+ of Sonos interactions via widget	Self-reported usage
Haptic response latency	< 100ms from tap to haptic	Manual testing
Visual state update latency	< 500ms from tap to UI refresh	Manual testing
Battery overhead	< 2% daily drain attributable to widget	Android battery stats
Widget crash rate	Zero unrecoverable states	Logcat monitoring
Speaker discovery time	< 2 seconds on local network	Timed local UPnP scan

4. User Personas & Scenarios

Since this is a personal project, the primary persona is the builder — but defining usage scenarios still helps prioritize features and edge cases.

4.1 Primary Persona: Owen

Tech-savvy Pixel 9 owner with a 3–5 room Sonos setup. Frequently plays music throughout the house while working, cooking, or entertaining. Values speed and minimal friction. Wants to glance at what's playing and make adjustments without breaking flow. Prefers visual polish — the widget should feel like a premium, intentional part of the home screen, not a utility afterthought.

4.2 Key Usage Scenarios

Scenario A: Quick Pause. The doorbell rings while music is playing in the Living Room. Owen glances at his phone on the desk, sees the widget showing the current track, and taps pause. Total interaction time: under 1 second. No unlock required if the widget is visible on the lock screen (stretch goal).

Scenario B: Volume Adjustment During a Call. Owen is on a phone call and background music is too loud. He swipes to the home screen, uses the volume slider on the widget to lower the Kitchen speaker to 20%. The slider provides haptic feedback and a numeric readout.

Scenario C: Party Mode Grouping. Guests are arriving. Owen opens the speaker grouping panel on the widget, selects Living Room, Kitchen, and Patio, and taps "Group." All three zones begin playing the same track in sync. Later, he ungroups the Kitchen when dinner conversation starts.

Scenario D: Queue Check and Skip. Owen wants to know what's coming up next. He scrolls the queue list in the full-size widget, sees a track he's not in the mood for, and taps skip to advance past it.

Scenario E: Source Switch. Currently playing a Spotify playlist, Owen wants to switch to his Jazz Chill playlist on Apple Music. He taps the source icon, selects Apple Music from the available services, and browses to the playlist — or taps through to the Sonos app for deeper browsing.

5. Feature Requirements

5.1 Feature Priority Matrix

Priority	Feature	Widget Size	Phase	Status
P0	Now Playing display (album art, track, artist)	All sizes	1	**DONE**
P0	Play / Pause toggle	All sizes	1	**DONE**
P0	Volume control with numeric readout (slider preferred; [–] % [+] buttons as fallback)	All sizes	1	**DONE**
P0	Skip forward / Skip back	All sizes	1	**DONE**
P0	Active speaker/zone selector	All sizes	1	**DONE**
P0	Adaptive color theming from album art	All sizes	1	**DONE**
P1	Speaker grouping / ungrouping	Half + Full	1	Ready
P1	Scrollable queue list	Full only	2	
P1	Shuffle / Repeat toggle	Half + Full	2	
P1	Music source switching	Full only	2	
P2	Click-through to Sonos app (deep link)	All sizes	1	**DONE**
P2	Static progress bar (updates on interaction/track change only)	Half + Full	3	
P2	Favorites / Presets quick-launch	Full only	3	
P2	Lock screen mini layout (dedicated compact widget)	Mini	3	

5.2 Feature Detail: Now Playing

- Displays album art as the dominant visual element, scaled to fit the widget.
- Track name displayed with single-line ellipsis truncation for long titles.
- Artist name displayed below track name, also truncated.
- Album art is fetched from the Sonos API (local or cloud) and cached in memory with a disk-backed LRU cache.
- When no music is playing, the widget shows the last-played track in a dimmed state with a “Resume” action.

5.3 Feature Detail: Speaker Grouping

- Displays all discovered Sonos zones as toggleable chips or checkboxes.
- Currently grouped speakers are pre-selected.
- User can add or remove speakers from the active group with a single tap.
- A “Group All” shortcut groups every discovered speaker.
- Grouping changes are executed immediately with optimistic UI updates.
- If grouping fails (e.g., speaker offline), the UI reverts with an inline error message.

5.4 Feature Detail: Queue Browsing

- Full-screen widget displays a scrollable list of upcoming tracks.
- Each queue item shows track name, artist, and a small thumbnail.
- Tapping a queue item jumps to that track (skip-to behavior).
- Queue automatically refreshes when playback state changes.
- For very long queues (50+ tracks), lazy loading with a “Load more” affordance.

5.5 Feature Detail: Music Source Switching

- Displays available music services linked to the Sonos account (Spotify, Apple Music, Amazon Music, Sonos Radio, etc.).
- Tapping a service shows recently played playlists or stations for that service.
- Selecting a playlist begins playback immediately.
- For deeper browsing (search, full library), a tap-through opens the Sonos app with a deep link to the relevant service.

6. Widget Layout & Sizing

6.1 Size Configurations

The widget supports dynamic resizing between three configurations, leveraging Jetpack Glance's SizeMode.Responsive:

Config	Grid	Approx Size	Content Displayed
Mini (Lock Screen)	3 × 1	~240 × 80 dp	Album art (tiny), track/artist, play/pause, skip fwd/back only
Half Screen	4 × 2	~320 × 180 dp	Album art (small), track/artist, play/pause, skip, volume, active zone selector
Full Screen	4 × 5	~320 × 480 dp	All half-screen content + speaker grouping panel, scrollable queue list, source switcher, shuffle/repeat

Note: The approximate dp sizes in this table are reference targets. Actual resize snap points depend on the Pixel 9 launcher's grid cell dimensions and may require on-device tuning during Phase 2 integration.

6.2 Half-Screen Layout (Compact)

The compact layout prioritizes immediate playback control in a tight horizontal space:

6. **Left zone (~40%):** Album art as a rounded-corner square, spanning the full height of the widget.
7. **Right zone (~60%):** Stacked vertically — track name (bold, single line), artist name (regular, single line), playback controls row (back, play/pause, forward), and a volume control (slider preferred; [–] % [+] buttons as fallback) with the active zone name.

Tapping the album art opens the Sonos app. Long-pressing the widget enters resize mode.

6.3 Full-Screen Layout (Expanded)

The expanded layout adds panels below the compact now-playing area:

8. **Top section:** Same as compact layout but with a larger album art (up to 120dp square) and room for a progress indicator.
9. **Controls bar:** Play/pause, skip back/forward, shuffle, repeat — arranged as a centered icon row.
10. **Volume section:** Full-width volume control with numeric percentage and active zone selector dropdown.
11. **Speaker grouping panel:** Horizontal scrollable row of zone chips (e.g., "Living Room," "Kitchen," "Office"). Tapping toggles group membership. "Group All" chip at the end.
12. **Queue section:** Scrollable vertical list showing next 10+ tracks with thumbnails. Occupies remaining vertical space.
13. **Source bar:** Bottom row showing current service icon. Tapping expands to show available services.

6.4 Mini Layout (Lock Screen)

A purpose-built hyper-compact layout designed for Android 16 lock screen widget constraints. Relying on OS scaling of the half-screen layout would produce cramped touch targets, unreadable text, and an unusable volume slider. The mini layout is a first-class citizen with its own composition:

14. **Left zone (~25%):** Album art as a small rounded square (48dp), which also serves as the reconnecting indicator surface.
15. **Center zone (~50%):** Track name (bold, single line, aggressive truncation) and artist name (single line) stacked vertically.
16. **Right zone (~25%):** Skip back, play/pause (primary, 48dp), and skip forward icons in a tight horizontal row.

No volume slider, speaker grouping, queue, or source switcher. Tapping any non-control area opens the Sonos app. This layout prioritizes glanceability and the single most common action: play/pause.

If the OS forces a bounding box smaller than the 3×1 grid, the widget degrades to an absolute minimum viable UI: album art and a single play/pause button. Skip back/forward buttons are hidden via a Glance size check, and the track/artist text is dropped.

7. Interaction Design

7.1 Touch Targets

All interactive elements must meet minimum 48dp × 48dp touch targets per Material Design guidelines. In the compact layout, play/pause is the largest target (56dp) as the most frequent action.

7.2 Gestures & Feedback

Due to Jetpack Glance's IPC-based rendering pipeline, visual state updates may take up to 500ms to appear. To maintain a perception of instant responsiveness, all interactive controls fire haptic feedback within 100ms of touch. The visual UI catches up asynchronously via the Glance recompose cycle.

Rich haptic feedback uses VibrationEffect.Composition with haptic primitives (API 31+) to create distinct tactile profiles for different control types: a soft pulse for play/pause, a sharp click for skip, and a gentle ramp for volume steps. On pre-API 31 devices, the system falls back to simple one-shot vibrations via HapticFeedbackConstants.

- **Single tap:** Primary action (play/pause, skip, toggle speaker, select source). Immediate haptic pulse on touch-down.
- **Slider drag / button tap:** Volume adjustment. If using a native Glance slider, provide real-time haptic feedback at 10% intervals. If using the [–] % [+] button fallback, each tap adjusts volume by 5% with a single haptic pulse. Long-press on [–] or [+] for continuous adjustment.
- **Scroll:** Vertical scroll for queue list; horizontal scroll for speaker chips.
- **Long press on widget:** Enter Android resize mode.
- **Tap on album art:** Launch the Sonos S2 app via getLaunchIntentForPackage. (A deep link to the current track's context within the Sonos app is a potential Phase 3 enhancement pending URI scheme availability.)

7.3 Optimistic Updates & Action Debouncing

All control actions update the widget UI immediately before the Sonos system confirms the state change. If the command fails (network timeout, speaker offline), the widget reverts to the last known good state and displays a brief inline error.

During cold-start reconnection (when the "Reconnecting…" indicator is visible), taps are queued but debounced: only the final intended state is executed once the service reconnects. For example, if the user taps Skip three times while disconnected, the widget executes a single skip-to-the-third-track command rather than three sequential skips.

A 3-second stale intent timeout applies to all debounced taps. If the service takes longer than 3 seconds to reconnect, all queued actions are silently discarded. This prevents ghost actions from executing against a stale playback state.

7.4 Loading & Empty States

- **Initial load:** Skeleton UI with pulsing placeholder shapes for album art, text, and controls.

- **No speakers found:** Friendly message with a “Retry” button and a hint to check Wi-Fi.
- **No music playing:** Dimmed last-played state with a prominent “Resume” button.
- **Network error:** Inline banner at the top of the widget, auto-dismissed after 5 seconds or on successful reconnection.

7.5 Layout Transitions

Jetpack Glance (and the underlying RemoteViews) does not support standard Jetpack Compose animations such as AnimatedVisibility or animateColorAsState. All layout transitions within the widget use simple visibility toggles — panels and overlays are either fully visible or fully gone, with no intermediate animation frames.

8. Visual Design System

8.1 Adaptive Album Art Theming

The SonosPlaybackService extracts a color palette from the current album art using Android's Palette API on a background coroutine whenever a track change is detected. The extraction runs in the service — never during widget recompose — to avoid ANRs and keep the Glance rendering path fast. The service downscales album art to a maximum of 200×200px before Palette processing to keep extraction under 20ms. The resulting pre-computed hex color values are written to the widget state, so the Glance composable only ever reads finished colors. The extracted colors are applied to:

- **Widget background:** A gradient or solid fill using the dominant dark color from the album art.
- **Text colors:** Light text derived from the vibrant or light-vibrant swatch, ensuring WCAG AA contrast against the background.
- **Swatch fallback:** If the target swatches (vibrant, light-vibrant) are null — common with monochromatic or low-contrast album art — the service computes the relative luminance of the dominant background color and selects pure white (luminance < 0.5) or pure black (luminance ≥ 0.5) for text and controls.
- **Controls:** Icon tint and slider accent color matched to the vibrant swatch.
- **Speaker chips:** Semi-transparent backgrounds tinted with the muted swatch.

When no album art is available, the widget falls back to a neutral dark theme (charcoal background, white text).

8.2 Typography

Element	Size	Weight	Notes
Track name	16sp	Bold	Single-line, ellipsis overflow
Artist name	13sp	Regular	Single-line, ellipsis overflow
Zone name	12sp	Medium	Displayed near volume control
Queue track	13sp	Regular	Two-line max (title + artist)
Volume %	11sp	Mono	Numeric readout beside slider

8.3 Iconography

Use Material Symbols (Outlined, weight 300) for all control icons: play_arrow, pause, skip_next, skip_previous, shuffle, repeat, volume_up, speaker_group, queue_music, and source-specific icons. Icons are tinted dynamically based on the adaptive color palette.

8.4 Corner Radius & Spacing

- **Widget outer radius:** System default (Android 12+ enforces rounded corners on widgets).
- **Album art radius:** 12dp.

- **Speaker chips radius:** 20dp (pill shape).
- **Internal padding:** 12dp on all edges; 8dp between elements.

9. Technical Architecture

9.1 High-Level Components

17. **SonosWidget (Jetpack Glance GlanceAppWidget):** The widget itself, defining responsive layouts for each size bucket and rendering UI from state.
18. **SonosWidgetReceiver (GlanceAppWidgetReceiver):** Handles widget lifecycle events (onUpdate, onEnabled, onDisabled) and triggers state refresh.
19. **SonosCompanionApp (Activity):** Minimal companion app for initial Sonos OAuth login, preference management, and manual widget refresh.
20. **SonosPlaybackService (Foreground Service):** A lightweight foreground service that subscribes to Sonos playback events (local UPnP eventing or cloud WebSocket), performs album art color extraction via the Palette API on track changes, and pushes fully-composed state updates (including pre-computed theme colors) to the widget via GlanceAppWidgetManager.
21. **SonosRepository (Kotlin class):** Abstraction layer that routes commands to the local controller or cloud controller based on network availability.
22. **LocalSonosController:** Handles UPnP/SSDP discovery, mDNS/Bonjour fallback discovery, and SOAP action calls over the local network.
23. **CloudSonosController:** Handles REST calls to the Sonos Cloud API with OAuth token management.
24. **ImageCache:** LRU cache for album art bitmaps with a 50MB hard disk cap (~500 images at ~100KB average). In-memory tier for hot entries with disk-backed overflow.

9.2 Module Structure & File Inventory

The project is organized as a single-module Android application with the root package com.sycamorecreek.sonoswidget. The following is the expected file inventory, provided as a reference for coding agents. Agents should create files incrementally per the task decomposition in Section 16, not all at once.

- `com.sycamorecreek.sonoswidget.app/` — SonosCompanionActivity.kt, SonosPlaybackService.kt, SonosApplication.kt (Hilt @HiltAndroidApp).
- `com.sycamorecreek.sonoswidget.widget/` — SonosWidget.kt (GlanceAppWidget), SonosWidgetReceiver.kt, SonosWidgetState.kt, SonosWidgetTheme.kt, layouts: CompactLayout.kt, ExpandedLayout.kt, MiniLayout.kt.
- `com.sycamorecreek.sonoswidget.sonos/` — SonosRepository.kt, SonosCommand.kt, SonosModels.kt.
- `com.sycamorecreek.sonoswidget.sonos.local/` — LocalSonosController.kt, SsdpDiscovery.kt, MdnsDiscovery.kt, SoapClient.kt, UpnpEventSubscriber.kt.
- `com.sycamorecreek.sonoswidget.sonos.cloud/` — CloudSonosController.kt, SonosOAuthManager.kt, SonosCloudApi.kt, TokenStore.kt.
- `com.sycamorecreek.sonoswidget.cache/` — AlbumArtCache.kt, PaletteExtractor.kt.
- `com.sycamorecreek.sonoswidget.di/` — AppModule.kt (Hilt @Module), ServiceModule.kt.
- `res/xml/` — sonos_widget_info.xml (AppWidgetProviderInfo).

- `res/layout/` — widget_preview.xml (static preview layout for picker).
- `res/values/` — strings.xml, themes.xml (Material 3 base theme for companion app).
- `AndroidManifest.xml` — Companion activity, service declaration (with android:foregroundServiceType="mediaPlayback"), receiver, permissions (INTERNET, ACCESS_WIFI_STATE, CHANGE_WIFI_MULTICAST_STATE, NEARBY_WIFI_DEVICES, FOREGROUND_SERVICE, FOREGROUND_SERVICE_MEDIA_PLAYBACK, POST_NOTIFICATIONS, VIBRATE).

CRITICAL: Android 16 (API 36) introduces Local Network Protections, gating LAN device access behind the NEARBY_WIFI_DEVICES runtime permission. Without requesting this at runtime before discovery, SSDP/mDNS/SOAP calls to Sonos speakers will silently fail.

- `res/xml/network_security_config.xml` — Permits cleartext traffic only to local network ranges (192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12).

9.3 Companion App (Minimal)

The companion app is a lightweight configuration surface, not a music control interface. It is built using standard Jetpack Compose with Material 3 components (not Glance). The agent has explicit permission to use androidx.compose.* imports within the .app package only — see Section 17.3 for the boundary rule. The companion app handles:

- Sonos OAuth 2.0 login: A single-screen flow using Android Custom Tabs.
- Manual speaker IP entry: A simple text input list for users whose routers block multicast discovery.
- Default zone selection: A dropdown populated from discovered speakers.
- Preferred music service: A picker for the default source shown in the widget's source switcher.
- Cache management: A "Clear image cache" button with current cache size display.
- Widget refresh: A manual "Refresh now" button for debugging connectivity issues.

No adaptive theming, no album art, no playback controls. Built with standard Jetpack Compose and Material 3. The app should be buildable in 1–2 days.

10. API & Authentication Strategy

10.1 Local Network (Primary)

The local controller uses UPnP/SSDP to discover Sonos devices on the LAN and communicates via SOAP actions over HTTP. This provides the lowest latency and works without internet access.

- **Local Network Permission (Android 16 requirement):** Before any local discovery or SOAP calls, the app must request the NEARBY_WIFI_DEVICES runtime permission. If the user denies the permission, fall back directly to the cloud API.
- **Discovery:** SSDP M-SEARCH for urn:schemas-upnp-org:device:ZonePlayer:1.
- **Control:** SOAP actions to AVTransport, RenderingControl, and GroupManagement services.
- **Events:** HTTP subscription (SUBSCRIBE) to receive real-time playback state change notifications.
- **Timeout:** 2-second timeout on local discovery; if no speakers found, fall back to cloud.
- **Multicast lock:** The service acquires a WifiManager.MulticastLock before SSDP discovery, held only during active discovery scans and released immediately after.
- **Concurrent SSDP + mDNS racing:** SSDP M-SEARCH and mDNS/Bonjour (NsdManager, querying _sonos._tcp.local) are fired concurrently using Kotlin coroutines. The first protocol to return a valid speaker result wins, and the other is cancelled. This caps the total local discovery phase at 2 seconds maximum.
- **Auto-detect IP fallback:** If concurrent SSDP/mDNS discovery fails, the companion app prompts the user to enter speaker IP addresses manually. Manually configured IPs are persisted in DataStore.

10.2 Cloud API (Fallback)

The Sonos Cloud API (api.ws.sonos.com) provides control when the phone is not on the same LAN as the speakers.

- **Authentication:** OAuth 2.0 Authorization Code flow. The companion app handles the initial login via Android Custom Tabs. The redirect URI is sonoswidget://callback, intercepted via an intent filter in AndroidManifest.xml.
- **Token storage:** Encrypted access and refresh tokens stored in Android Keystore-backed EncryptedSharedPreferences.
- **Token refresh:** Automatic refresh when access token expires (typically 24 hours).
- **Rate limits:** Sonos Cloud API has rate limits; the widget batches and throttles commands to stay well within limits.

10.3 Fallback Strategy

The local discovery chain follows a tiered approach before resorting to the cloud:

25. SSDP and mDNS fire concurrently via Kotlin coroutines (2-second timeout). First success wins; the other is cancelled.

26. If both SSDP and mDNS fail: attempt direct connection to any manually configured speaker IPs persisted in DataStore.
27. If all local methods fail (or phone is on cellular): seamlessly switch to the Sonos Cloud API.

On every command or state refresh, SonosRepository walks this chain. The active channel (SSDP, mDNS, manual IP, or cloud) is cached for 30 seconds to avoid repeated discovery attempts. A small icon on the widget indicates the active connection mode.

11. Data Flow & State Management

11.1 Widget State Model

The widget's UI is driven by a single immutable data class (SonosWidgetState) persisted in Glance's DataStore. The following is the canonical shape — coding agents should implement this exactly:

- **playbackState:** enum PlaybackState { PLAYING, PAUSED, STOPPED, TRANSITIONING }
- **currentTrack:** data class Track(val name: String, val artist: String, val album: String, val artUrl: String?, val durationMs: Long, val elapsedMs: Long)
- **activeZone:** data class Zone(val id: String, val displayName: String, val groupId: String?, val isGroupCoordinator: Boolean)
- **volume:** Int (0–100)
- **zones:** List<Zone> — All discovered zones with their current group membership.
- **queue:** List<QueueItem> — data class QueueItem(val trackName: String, val artist: String, val thumbnailUrl: String?, val position: Int). First 20 cached; lazy load beyond.
- **availableSources:** List<MusicSource> — data class MusicSource(val id: String, val name: String, val iconResId: Int)
- **connectionMode:** enum ConnectionMode { LOCAL_SSDP, LOCAL_MDNS, LOCAL_MANUAL_IP, CLOUD, DISCONNECTED }
- **shuffleEnabled:** Boolean
- **repeatMode:** enum RepeatMode { NONE, ALL, ONE }
- **colorPalette:** data class WidgetColorPalette(val background: String, val textPrimary: String, val textSecondary: String, val accent: String, val chipBackground: String) — all hex strings, pre-computed in SonosPlaybackService via Palette API. Widget never runs Palette directly.
- **isReconnecting:** Boolean — true when service is cold-starting after idle teardown.
- **isRateLimited:** Boolean — true when cloud API returns 429.
- **lastUpdatedMs:** Long — timestamp of last successful state sync, used for stale intent timeout (3-second window).

Phase 1 implementation note: The actual implementation serializes the full SonosWidgetState (including nested Track, Zone[], WidgetColorPalette) as a single JSON string in Glance Preferences via WidgetStateStore, rather than flattening 30+ fields into individual DataStore keys. This is architecturally equivalent and avoids key proliferation.

11.2 Update Flow

28. SonosPlaybackService receives a playback event (UPnP notification or cloud WebSocket).
29. If the track changed, the service fetches album art, downscales to 200×200px, and runs Palette API extraction on a background coroutine to produce theme colors.
30. Service maps the event (and pre-computed colors) to a SonosWidgetState update.
31. Service calls GlanceAppWidgetManager.updateAll() to trigger a recompose.

32. The Glance composable reads the updated state and renders the new UI.

For user-initiated actions (tap play, adjust volume), the widget dispatches an ActionCallback that tells the service to execute the command and optimistically updates state.

Phase 1 implementation note: Album art is written to disk as album_art_current.webp by the service. The Glance provideGlance() reads this file via ImageProvider. A lastArtUrl field avoids redundant fetches when the track hasn’t changed.

12. Performance & Battery Budget

12.1 Hybrid Update Strategy

Active Mode (music is playing)

- A lightweight foreground service runs with a minimal persistent notification (“SonosWidget: Connected”).
- The service subscribes to UPnP eventing (local) or maintains a WebSocket (cloud) for push-based updates.
- Playback state changes (track change, pause, volume) arrive in near-real-time (under 1 second).
- Album art is pre-fetched on track change, not on widget recompose.

Idle Mode (no music playing)

- The foreground service is torn down after 5 minutes of silence to conserve battery.
- The widget falls back to periodic WorkManager updates every 15 minutes (Android minimum).
- When the user taps any control on the widget, the service is re-launched instantly.

12.2 Battery Targets

Scenario	Target Drain	Mechanism
Active playback (4 hrs/day)	< 1.5% battery	Foreground service + push events
Idle (20 hrs/day)	< 0.5% battery	WorkManager periodic only
Album art fetch per track	< 50ms + 100KB avg	LRU cache, JPEG compression
Palette extraction per track	< 20ms (200×200px input)	Downscale before extraction; run in service

13. Error Handling & Edge Cases

13.1 Error Scenarios

Scenario	Widget Behavior	Recovery
Wi-Fi disconnected	Switch to cloud; show cloud icon	Auto-switch back when LAN returns
No internet + no LAN	Show “Offline” badge; disable controls	Retry on network change broadcast
Speaker powered off	Gray out speaker chip; inline notice	Re-discover on next poll cycle
OAuth token expired	Silent refresh; if fails, prompt re-login via companion app	Companion app handles re-auth
Command timeout (>3s)	Revert optimistic update; show error	User can retry; auto-retry once
Album art unavailable	Show generic music icon with neutral theme	Retry on next track change
Sonos firmware update	Show “Updating…” badge; disable controls	Resume normal operation post-update
Cloud API rate limited	Show “Rate Limited” pill badge on album art; disable controls temporarily	Auto-retry after backoff period (30s); badge auto-dismissed on success
Cold start after idle teardown	Show last-known state with semi-transparent “Reconnecting…” pill badge overlaid on album art; controls remain tappable (debounced)	Service relaunches on tap; UI updates when connected
Local network permission denied	Skip all local discovery; connect via cloud API only. Show cloud icon and a one-time inline hint: “Local control unavailable — grant network permission in Settings for faster response”	Monitor permission state; re-attempt local discovery if permission is later granted

13.2 Defensive Design Principles

- **Never crash the widget.** All exceptions caught and mapped to error states.
- **Never show a blank widget.** Always fall back to last-known-good state or a meaningful empty state.
- **Never block the UI thread.** All Sonos communication is asynchronous (coroutines).
- **Never leak memory.** Album art bitmaps are recycled aggressively; the LRU cache enforces a 50MB hard disk cap with automatic eviction.

14. Accessibility

- All controls have contentDescription labels (e.g., “Pause playback,” “Volume: 45%,” “Group Living Room speaker”).
- Text contrast meets WCAG AA (4.5:1 minimum) against the adaptive background. The theme extraction algorithm validates contrast ratios and adjusts if needed.
- Touch targets are minimum 48dp × 48dp.
- The volume slider supports TalkBack with a step announcement (“Volume 45%, double tap to adjust”).
- Speaker chips announce their group state (“Living Room, grouped” or “Kitchen, ungrouped”).

15. Security Considerations

- OAuth tokens are stored in EncryptedSharedPreferences backed by Android Keystore. Never written to plain SharedPreferences, logs, or disk.
- Local UPnP communication is HTTP by protocol design. Acceptable because it never leaves the LAN. Network Security Config restricts cleartext to local network IP ranges.
- NEARBY_WIFI_DEVICES runtime permission is requested before any LAN access. If denied, the app degrades gracefully to cloud-only mode.
- Cloud API calls use HTTPS exclusively with certificate pinning for the Sonos API domain.
- No analytics, telemetry, or third-party SDKs. Zero data transmission beyond Sonos API calls.
- Single-household scoping: All OAuth tokens, discovery caches, and speaker configurations are scoped to one Sonos system.
- ProGuard/R8 obfuscation is optional (personal use) but recommended for good practice.

16. Agent Build Plan & Task Decomposition

This project is implemented by coding agents (e.g., Claude Code). Each phase is broken into discrete, atomic tasks sized to fit within a single agent context window. Tasks are sequentially ordered — each builds on the output of the previous. Between tasks, the human owner verifies the build compiles and the acceptance criteria are met before handing the next task to the agent.

Key principles for agent-driven execution: (1) Each task produces a compilable, testable increment. (2) Tasks specify exact files to create or modify. (3) Acceptance criteria are binary (pass/fail). (4) No task requires the agent to hold more than one phase's worth of context. (5) When starting a new task, the agent should read this PRD and the relevant existing source files as context.

Phase 1: Project Scaffolding & Local Playback (8 Tasks) — COMPLETE

Outcome: A working widget on the home screen that discovers a local Sonos speaker, displays now-playing info, and provides play/pause, skip, and volume control. All 8 tasks compiled with zero warnings and zero errors.

Task 1.1: Project Bootstrap — COMPLETE (v1.9)

Create the Android project from scratch. Set up Gradle (Kotlin DSL) with the version catalog from Section 17, apply Hilt, configure compileSdk 36 / minSdk 35 / targetSdk 36, and add all dependencies. Configure cleartext HTTP traffic for local UPnP communication.

- **Files created:** build.gradle.kts (project + app), settings.gradle.kts, gradle.properties, libs.versions.toml, AndroidManifest.xml (with network_security_config.xml restricting cleartext to local IP ranges), SonosApplication.kt.
- **Build environment:** CLI and agent builds require JAVA_HOME and ANDROID_HOME to be set before running ./gradlew. The project uses KSP (not kapt) for annotation processing.

Task 1.2: Widget Shell — COMPLETE (v2.0)

Create the Glance widget with a hardcoded placeholder layout. Register the widget receiver and provider info. Register a static android:previewLayout in sonos_widget_info.xml showing a representative now-playing UI for the widget picker.

- **Files created:** SonosWidget.kt, SonosWidgetReceiver.kt, SonosWidgetState.kt, sonos_widget_info.xml, res/layout/widget_preview.xml, CompactLayout.kt.
- **Decision E8:** Widget picker preview uses android:previewLayout (standard Android 12+ XML mechanism), not the unverified Glance 1.2 providePreview() API. See Section 19.

Task 1.3: SSDP + mDNS Discovery — COMPLETE

Implement concurrent local speaker discovery. Fire SSDP and mDNS simultaneously using coroutines; first success wins within a 2-second timeout.

- **Files created:** SsdpDiscovery.kt, MdnsDiscovery.kt, LocalSonosController.kt (discovery orchestrator, permission gate).
- **Note:** SSDP returns richer data (displayName, modelName, UDN) than mDNS (service name + IP/port). Repository layer may optionally do a follow-up XML fetch for

mDNS-discovered speakers. mDNS uses the API 34+ modern path (registerServiceInfoCallback + hostAddresses).

Task 1.4: SOAP Control Client — COMPLETE

Implement SOAP action calls for AVTransport (play, pause, stop, next, previous, seek, getTransportInfo, getPositionInfo, getMediaInfo) and RenderingControl (get/set volume, get/set mute). Implement ZoneGroupTopology:getZoneGroupState with entity-encoded XML parsing.

- **Files created:** SonosSoapClient.kt (SOAP transport + XML helpers), SoapModels.kt (typed response data classes), SonosControlActions.kt (high-level typed wrappers).
- **Key addition:** pollPlaybackState() on LocalSonosController batches getTransportInfo, getPositionInfo, and getVolume concurrently via coroutineScope + async. Use this for initial state fetch and fallback polling in the service layer.

Task 1.5: Playback Service + UPnP Event Subscription — COMPLETE

Create the foreground service that subscribes to UPnP events and pushes state updates to the widget. Implement the hybrid active/idle strategy with 5-minute teardown.

- **Files created:** PlaybackService.kt (foreground service with adaptive polling), WidgetStateMapper.kt, WidgetStateStore.kt (JSON serialization to Glance Preferences), UpnpEventSubscriber.kt.
- **CRITICAL:** foregroundServiceType="mediaPlayback" declared in manifest + FOREGROUND_SERVICE_MEDIA_PLAYBACK permission. Without both, startForeground() throws SecurityException on API 34+.

Task 1.6: SonosRepository + Widget Integration — COMPLETE

Wire the service, controller, and widget together through SonosRepository. Widget reads state from DataStore and renders real now-playing info.

- **Files created:** SonosRepository.kt (interface + impl), SonosPreferences.kt (DataStore prefs), WidgetActions.kt (Glance ActionCallbacks), AppModule.kt (Hilt bindings).
- **Volume control:** Implemented as [–] % [+] button row with 5% step increments and haptic feedback per tap. Native Glance slider was evaluated but the button fallback provides more reliable interaction.

Task 1.7: Album Art + Adaptive Theming — COMPLETE

Fetch album art, cache it, and extract colors in the service. Wire pre-computed palette colors into the widget background and text.

- **Files created:** AlbumArtLoader.kt (Coil ImageLoader.execute → Bitmap pipeline), ThemeExtractor.kt (Palette API extraction).
- **Coil-to-Glance pipeline:** Album art is written to disk as album_art_current.webp by the service. provideGlance() reads this file via ImageProvider(bitmap). Standard Coil AsyncImage composables do NOT work in Glance.

Task 1.8: Controls + Zone Selector + Deep Link — COMPLETE

Implement skip forward/back ActionCallbacks, active zone selector (chip row), haptic feedback on all controls, and Sonos app launch on album art tap.

- **Files created:** HapticHelper.kt (three haptic profiles: confirm, click, ramp).

- **Haptics:** VibrationEffect.Composition with haptic primitives on API 31+, with simple one-shot vibration fallback. See decision E10 in Section 19.
- **Deep link:** Launches Sonos S2 app (com.sonos.acr2) via getLaunchIntentForPackage(). A URI-scheme deep link to the current track is a potential Phase 3 enhancement.
- **Zone chips:** Compact widget shows up to 4 zone chips (group coordinators only) with “+N” overflow indicator.

Phase 2: Full Screen, Grouping & Cloud (7 Tasks)

Outcome: Full-screen layout with speaker grouping, queue browsing, shuffle/repeat. Cloud API fallback operational.

Task 2.1: Full-Screen Layout Shell

Build the ExpandedLayout composable that includes all compact content plus placeholder sections for grouping, queue, and source bar.

- **Files to create:** ExpandedLayout.kt. Modify SonosWidget.kt to switch layouts based on SizeMode.Responsive size classes.
- **Cleanup opportunity:** Consolidate the duplicate extractXmlValue helper (exists in both SsdpDiscovery.kt and SonosSoapClient.kt) into a shared utility.
- **Acceptance:** Resizing the widget to full screen shows the expanded layout with clearly delineated sections. Compact content still renders correctly in the top area.

Task 2.2: Speaker Grouping

Implement grouping/ungrouping via UPnP GroupManagement SOAP calls. Build the speaker chip row UI with optimistic updates.

- **Files to modify:** LocalSonosController.kt (add group/ungroup SOAP), ExpandedLayout.kt (chip row), CompactLayout.kt (chip row for half-screen), SonosWidgetState.kt (group membership).
- **Acceptance:** Tapping a speaker chip toggles it in/out of the active group. “Group All” groups every speaker. Ungrouping a speaker removes it. Optimistic UI updates with revert on failure.

Task 2.3: Scrollable Queue

Implement the scrollable queue list using Glance’s LazyColumn equivalent. Each item shows track name, artist, and small thumbnail. Tapping jumps to that track.

- **Files to modify:** ExpandedLayout.kt (queue section), SonosWidgetState.kt (queue field), LocalSonosController.kt (get queue SOAP), SonosPlaybackService.kt (queue refresh on track change).
- **Acceptance:** Queue list scrolls smoothly with 20+ items. Tapping a queue item starts playing that track. Queue refreshes when tracks change. Thumbnails load and display.

Task 2.4: Shuffle & Repeat

Add shuffle and repeat toggle buttons to the controls bar. Wire to SOAP actions.

- **Acceptance:** Shuffle toggle changes visual state and Sonos shuffle mode. Repeat cycles through NONE → ALL → ONE. State persists across widget updates.

Task 2.5: OAuth Flow + Cloud API Client

Build the companion app’s OAuth login screen and the Cloud API client. Store tokens securely.

- **OAuth redirect (CRITICAL):** Configure an <intent-filter> in AndroidManifest.xml for SonosCompanionActivity to intercept the custom scheme redirect URI sonoswidget://callback. The intent filter needs: <action android:name="android.intent.action.VIEW" />, <category

android:name="android.intent.category.DEFAULT" />, <category android:name="android.intent.category.BROWSABLE" />, and <data android:scheme="sonoswidget" android:host="callback" />.

- **Files to create:** SonosCompanionActivity.kt, SonosOAuthManager.kt, TokenStore.kt, SonosCloudApi.kt, CloudSonosController.kt. Modify AndroidManifest.xml.
- **Acceptance:** Tapping “Sign in with Sonos” opens Custom Tab, completes OAuth, redirects back via sonoswidget://callback, and stores tokens in EncryptedSharedPreferences. CloudSonosController can call getPlaybackStatus and return the current track.

Task 2.6: Fallback Router

Implement the concurrent(SSDP, mDNS) → manual IP → cloud fallback chain in SonosRepository. Add connection mode icon to widget.

- **Acceptance:** Disabling Wi-Fi causes widget to switch to cloud control within 5 seconds. Re-enabling Wi-Fi switches back to local. Connection icon updates accordingly.

Task 2.7: Companion App Settings

Build the remaining companion app screens: manual IP entry, default zone picker, preferred service picker, cache management.

- **Acceptance:** All settings save and persist across app restarts. Manual IPs validated with a connection test. “Clear cache” shows size and clears the 50MB disk cache. “Refresh now” triggers a widget update.

Phase 3: Sources, Lock Screen & Polish (8 Tasks)

Outcome: Music source switching, lock screen mini layout, error handling, debouncing, and all edge cases resolved.

Task 3.1: Music Source Listing

Implement available music service discovery and recent playlist retrieval from the Sonos Cloud API.

- **Acceptance:** Source bar shows linked services. Tapping a service shows recently played playlists. Selecting a playlist starts playback.

Task 3.2: Mini Lock Screen Layout

Build the MiniLayout composable (3×1 grid) with extreme compression fallback (album art + play/pause only).

- **Acceptance:** Mini layout renders correctly at ~240×80dp. Under extreme compression, only album art + play/pause remain.

Task 3.3: Error States & Status Badges

Implement all error scenarios from Section 13.1. Build the semi-transparent pill badge component for “Reconnecting…” and “Rate Limited” overlays on album art. Use Glance 1.2’s image alpha API for badge transparency.

- **Acceptance:** Each error scenario from Section 13.1 produces the correct widget behavior. Pill badges overlay correctly on album art. Badges auto-dismiss on recovery.

Task 3.4: Action Debouncing & Stale Intent

Implement the debounce queue for cold-start taps and the 3-second stale intent timeout.

- **Acceptance:** Tapping Skip 3x during “Reconnecting…” results in a single skip-3 command when connected. Taps older than 3 seconds are silently discarded.

Task 3.5: Static Progress Bar

Add a non-animated progress bar that updates only on track change and user interaction.

- **Acceptance:** Progress bar shows correct position on track change. Does not animate between updates.

Task 3.6: Accessibility Pass

Add contentDescription to all interactive elements. Validate contrast ratios. Ensure TalkBack support.

- **Acceptance:** TalkBack reads meaningful labels for every control. No contrast violations against adaptive backgrounds.

Task 3.7: Battery & Performance Optimization

Profile battery usage, optimize service lifecycle, tune cache eviction, verify idle teardown, and stress-test queue scrolling.

- **Acceptance:** < 2% daily battery drain per Android battery stats. Service tears down cleanly after 5 min idle. Queue scrolls smoothly with 20+ items. No memory leaks.

Task 3.8: Final Integration Test

End-to-end validation of all features across all three widget sizes.

- **Acceptance:** All features from Section 5.1 work as specified. All error states from Section 13.1 are handled. Discovery chain works. OAuth flow completes. Widget survives device reboot.

17. Tech Stack & Dependency Manifest

All dependencies should be added in Task 1.1 using the version catalog below. CRITICAL for coding agents: Do not guess dependency versions. Use these exact versions, or if any fails to resolve, search Maven Central for the latest stable release rather than guessing.

17.1 Version Catalog (libs.versions.toml)

The agent should create this file at gradle/libs.versions.toml during Task 1.1. All version numbers below are pinned. The agent must use these exact versions and must NOT substitute its own guesses:

[versions] kotlin = "2.1.21", agp = "8.10.1", compileSdk = "36", minSdk = "35", targetSdk = "36", glance = "1.2.0-rc01", hilt = "2.58", ksp = "2.1.21-2.0.1", coroutines = "1.9.0", okhttp = "4.12.0", retrofit = "2.11.0", coil = "2.7.0", datastore = "1.1.1", palette = "1.0.0", security-crypto = "1.1.0-alpha06", work = "2.10.0", material = "1.12.0", compose-bom = "2024.12.01", activity-compose = "1.9.3", core-ktx = "1.15.0", lifecycle-runtime-ktx = "2.8.7"

[libraries] glance-appwidget, glance-material3, hilt-android, hilt-compiler, coroutines-android, okhttp, retrofit, coil, datastore-preferences, palette, security-crypto, work-runtime, material, compose-bom, activity-compose, core-ktx, lifecycle-runtime-ktx (see v1.9 PRD Section 17.1 for full group/name/version.ref mappings)

[plugins] android-application (AGP), kotlin-android, hilt, compose-compiler, ksp

17.2 Component Summary

Component	Technology
Language	Kotlin 2.1.21 (100%)
Compile / Target SDK	API 36 (Android 16)
Min SDK	**API 35 (Android 15) — Pixel 9. This is enforced. Do not lower.**
Widget Framework	Jetpack Glance 1.2.0-rc01 (androidx.glance:glance-appwidget)
Widget UI Toolkit	Glance Composables only — NOT Jetpack Compose
Companion App UI	Standard Jetpack Compose (androidx.compose.*) — see Section 17.3
Async	Kotlin Coroutines 1.9.0
Networking	OkHttp 4.12.0 + Retrofit 2.11.0
Image Loading	Coil 2.7.0 (via ImageLoader.execute → Bitmap pipeline)
Color Extraction	AndroidX Palette 1.0.0
State Persistence	DataStore Preferences 1.1.1 (JSON serialization via WidgetStateStore)
Dependency Injection	Hilt 2.58 (via KSP)
Token Storage	EncryptedSharedPreferences (security-crypto 1.1.0-alpha06)

Background Work	Foreground Service (active) + WorkManager 2.10.0 (idle)
Haptics	VibrationEffect.Composition with haptic primitives (API 31+), one-shot vibration fallback
Build System	Gradle 8.11.1 (Kotlin DSL), AGP 8.10.1
Target Device	Pixel 9, Android 16

17.3 Companion App UI Toolkit Clarification

The strict prohibition on using androidx.compose.* (Section 20.4) applies ONLY to widget code in the com.sycamorecreek.sonoswidget.widget package. The companion app (com.sycamorecreek.sonoswidget.app) should be built using standard Jetpack Compose with Material 3 components.

Import boundary nuance: Glance itself depends on certain Compose value types (e.g., androidx.compose.ui.graphics.Color, androidx.compose.ui.unit.dp, androidx.compose.ui.unit.sp). Using these value types in widget code is permitted because they are transitive dependencies of the Glance library, not Compose UI composables. The prohibition is specifically against importing Compose UI toolkit composables (e.g., androidx.compose.material3.*, androidx.compose.foundation.layout.*, or any @Composable functions from the Compose runtime) in widget files. If in doubt: if the import resolves to a data type or constant that Glance's own API surface uses, it is safe.

18. Open Questions & Risks

18.1 Open Questions (Remaining)

33. **Sonos Cloud API WebSocket support:** The Sonos API documentation for WebSocket-based real-time updates may have changed. Needs verification against the latest developer docs.
34. **UPnP event subscription reliability:** SUBSCRIBE-based notifications can be dropped if the phone's Wi-Fi enters power-saving mode. May need a periodic poll fallback even in active mode.
35. **Music source API depth:** The Sonos Cloud API may have limitations on browsing third-party service libraries (Spotify, Apple Music). The widget may need to fall back to deep-linking for some browse actions.
36. **Production API key provisioning:** Timeline and requirements for obtaining a production Sonos Developer client ID need to be confirmed before Phase 2 cloud integration begins.
37. **Sonos S2 deep link URI scheme:** Phase 1 launches the Sonos app generically. If the PRD's "deep link to current track's context" vision is desired for Phase 3, research is needed into whether Sonos S2 exposes a URI scheme (e.g., sonos://now-playing). Low priority.

18.2 Key Risks

Risk	Severity	Mitigation
Sonos API deprecation or changes	High	Abstract behind SonosRepository; local control works independent of cloud API
Glance rendering limitations	Medium	Queue list must be optimized aggressively; visibility toggles replace animations
Battery drain from foreground service	Medium	Aggressive idle teardown; profiling in Phase 3
UPnP discovery flakiness on Wi-Fi 6E	Low–Med	Cloud fallback; cache discovered speakers for 5 min
Scope creep across phases	Low	Strict phase gating; P2/P3 features deferred until core is solid
Agent context window limits	Medium	Tasks decomposed to single-file or small-file-group scope; PRD available as reference
Agent-generated code drift	Medium	Verify each task compiles before starting next; PRD is source of truth for architecture. minSdk 35 is enforced (see E9).
UPnP/SOAP protocol complexity	Medium	Agent should reference Sonos UPnP documentation and use raw HTTP/XML

19. Appendix: Engineering Review Decision Log

The following decisions were made during engineering, design, and product review cycles and are incorporated throughout the document. Decisions are numbered sequentially within each category.

19.1 Engineering Decisions

E1: Latency Metric Split

Question: Glance IPC latency means visual state updates may exceed 300ms.

Decision: Split into two metrics: < 100ms for haptic feedback, < 500ms for visual state update. Haptic response is the primary perceived-speed signal.

E2: Progress Bar — Static Snapshots Only

Decision: Progress bar updates only on user interaction and track changes. No continuous animation.

E3: Local Discovery — Concurrent SSDP + mDNS Racing

Decision: Race SSDP and mDNS concurrently using Kotlin coroutines with a shared 2-second timeout. First protocol to return a valid speaker result wins; the other is cancelled.

E4: Action Debouncing During Cold Starts

Decision: Debounce: only the final intended state is executed. Repeated identical actions are collapsed.

E5: Image Cache — 50MB Hard Disk Cap

Decision: 50MB. At ~100KB per compressed album art JPEG, this stores approximately 500 cached images.

E6: Stale Intent Timeout — 3-Second Discard Window

Decision: If the service takes longer than 3 seconds to reconnect after a cold start, all queued debounced taps are silently discarded.

E7: Hilt 2.52 → 2.58 and KSP Adoption (Task 1.1 Finding)

Decision: Upgrade Hilt from 2.52 to 2.58 and adopt KSP (2.1.21-2.0.1) as the annotation processing backend, replacing the deprecated kapt. Hilt 2.58 is the latest version compatible with AGP 8.10.x (2.59+ requires AGP 9.0.0).

E8: Widget Preview — previewLayout over providePreview() (Task 1.2 Finding)

Question: Glance 1.2.0-rc01 is specified in the version catalog, but the providePreview() method signature cannot be confirmed against the published artifact.

Decision: Use android:previewLayout in sonos_widget_info.xml (the standard Android 12+ / API 31 mechanism). The XML preview layout renders a static now-playing UI in the widget picker. This is functionally equivalent to a composable preview for a static representation.

Contingency: If Glance 1.2.0 stable ships with a confirmed providePreview() API, migration from XML preview is trivial. No architecture changes required.

E9: minSdk 35 Enforcement (Phase 1 Completion Finding)

Question: During Phase 1 implementation, the agent lowered minSdk from 35 to 28, adding version-branching fallback code for pre-API 31 haptics and other features. Should the broader device compatibility be retained?

Decision: Revert to minSdk 35. This is a personal-use project targeting a single Pixel 9 running Android 16. minSdk 35 eliminates all version-branching code, avoids dead fallback paths, and keeps the codebase clean. The agent must not lower minSdk without explicit product approval. If minSdk needs to change in the future, it is a PRD-level decision, not an agent implementation choice.

Impact: Section 17.1 version catalog, Section 17.2 component summary. Agents should remove any Build.VERSION.SDK_INT checks that branch below API 35 and delete unreachable fallback code. The haptics implementation should use VibrationEffect.Composition directly (available since API 31) without wrapping in a version check.

E10: Haptics — API 31+ Composition Primitives, Not Android 16-Specific (Phase 1 Completion Finding)

Question: The PRD referenced “Android 16’s richer haptics APIs (custom amplitude/frequency curves)”. Does Android 16 (API 36) introduce new haptic primitives?

Decision: No. VibrationEffect.Composition with haptic primitives (PRIMITIVE_CLICK, PRIMITIVE_TICK, PRIMITIVE_LOW_TICK, etc.) was introduced in API 30–31. Android 16 does not add new haptic primitives beyond what API 31–35 established. The PRD’s Android 16 haptics reference was incorrect. The Phase 1 implementation correctly uses API 31+ composition primitives with three distinct profiles (confirm, click, ramp). Since minSdk is 35, these APIs are always available with no version check needed.

19.2 Design Decisions

D1: Layout Transitions — Visibility Toggles

Decision: All transitions use simple visibility toggles (fully visible or fully gone). No intermediate animation frames.

D2: Color Extraction — Background Service Processing

Decision: All Palette API extraction runs in SonosPlaybackService on a background coroutine. Album art is downscaled to 200×200px. Widget receives only pre-computed hex color values.

D3: Queue List — Full Scrollable, Optimized

Decision: Commit to the full scrollable queue list with optimization. The queue is a core differentiator over the media notification.

D4: Lock Screen — Dedicated Mini Layout

Decision: Build a purpose-built hyper-compact “mini” layout (3×1 grid, ~240×80dp). OS scaling the half-screen layout produces unpredictable results.

D5: Reconnecting Indicator — Album Art Pill Badge

Decision: A small, semi-transparent pill badge overlaid on the album art. Same pattern reused for “Rate Limited” state.

D6: Palette Swatch Failures — Luminance-Based Text Selection

Decision: When target swatches are null, compute relative luminance of the dominant background color and select pure white (luminance < 0.5) or pure black (luminance ≥ 0.5).

D7: Extreme Lock Screen Compression — Album Art + Play/Pause Minimum

Decision: Under extreme compression, drop skip buttons and track/artist text. Show only album art and play/pause.

19.3 Product Decisions

P1: Sonos API — Production Client ID

Decision: A production client ID will be obtained from the Sonos Developer portal. Timeline tracking in Section 18.1.

P2: Cold Start After Idle — Last-Known State with Reconnecting Indicator

Decision: Widget displays last-known playback state with a “Reconnecting…” pill badge. Controls remain tappable but debounced.

P3: Cloud API Rate Limiting — Specific Badge

Decision: Show a specific “Rate Limited” pill badge. Auto-dismisses after 30-second backoff. Controls temporarily disabled.

P4: Multi-Household — Single Household Only

Decision: Single household only. SonosRepository interface is the natural extension point if ever needed.

P5: Companion App UX — Bare-Bones Material 3

Decision: Bare-bones Material 3 settings list. No adaptive theming, no playback controls. Estimated build time: 1–2 days.

20. Appendix: Agent Workflow Guidelines

This section provides guidance for coding agents executing the build tasks in Section 16.

20.1 Context Management

- At the start of every task, the agent should read: (1) this PRD (or the relevant sections), (2) any existing source files it will modify, and (3) the build.gradle.kts for dependency context.
- The agent should not attempt to hold the entire codebase in context. Each task is scoped to a small set of files. Trust the task boundaries.
- If a task requires understanding a class the agent didn't create, the agent should read that file first before making changes.
- When this PRD is too large to fit alongside code in a single context window, prioritize: (1) the current task description from Section 16, (2) the relevant architecture section (9.x), (3) the state model (Section 11.1), (4) the specific feature detail from Section 5.x.

20.2 Task Execution Protocol

38. Read the task description, files to create/modify, and acceptance criteria.
39. Read any existing files listed in "Files to modify."
40. Implement the changes. Prefer small, focused commits over large sweeping changes.
41. Ensure the code compiles: run ./gradlew assembleDebug and fix any errors before declaring the task complete.
42. Self-check against the acceptance criteria. If any criterion is ambiguous, make the conservative choice and note the assumption.

20.3 Code Style Conventions

- **Language:** Kotlin only. No Java files.
- **Formatting:** Follow the default ktlint rules. 4-space indentation, no wildcard imports.
- **Naming:** Standard Kotlin conventions. PascalCase for classes, camelCase for functions and properties, SCREAMING_SNAKE_CASE for constants.
- **Coroutines:** Use structured concurrency (viewModelScope, lifecycleScope, or supervisorScope). Never use GlobalScope.
- **Error handling:** Never throw unhandled exceptions in widget or service code. Wrap all Sonos API calls in try/catch and map failures to SonosWidgetState error fields.
- **Data classes:** Keep data classes in dedicated files (SonosModels.kt, SonosWidgetState.kt). Do not scatter data classes across multiple files.
- **String literals:** No hardcoded user-facing strings. Use strings.xml for anything visible in the widget or companion app.

20.4 Glance-Specific Pitfalls

Coding agents should be aware of these Glance-specific constraints that differ from standard Jetpack Compose:

- **IMPORT BOUNDARY:** Glance composables run in a different runtime than Jetpack Compose. In ALL files inside the .widget package, do not import Compose UI toolkit composables from androidx.compose.* (e.g., material3, foundation.layout, or Compose runtime @Composable functions). Compose value types that Glance itself depends on (Color, dp, sp) are permitted. See Section 17.3 for the full boundary rule. EXCEPTION: The companion app (.app package) uses standard Jetpack Compose.
- **ANNOTATION PROCESSING:** This project uses KSP (not kapt) for annotation processing. Hilt and all future annotation processors must be wired via ksp() in dependencies, not kapt().
- **MINSDK ENFORCEMENT:** minSdk is 35. Do not lower this. Do not add Build.VERSION.SDK_INT checks that branch below API 35. All APIs available on API 31+ (haptics, NsdManager modern path, etc.) are always available. See decision E9.
- No remember, no mutableStateOf, no LaunchedEffect in widget code. State is read from GlanceStateDefinition (DataStore), not from in-memory Compose state.
- No Modifier.clickable(). Use GlanceModifier.clickable(actionRunCallback<MyCallback>()) with ActionCallback classes.
- No Canvas, no CustomLayout, no AnimatedVisibility. Only the primitives Glance provides: Row, Column, Box, Text, Image, LazyColumn, Button, Spacer, and their Glance variants.
- **Images:** Must be passed as android.graphics.Bitmap via ImageProvider(bitmap). Do NOT use Coil's AsyncImage or any Compose-based image loader inside Glance layouts. The service writes the bitmap to album_art_current.webp on disk; provideGlance() reads it.
- **Widget picker preview:** Use android:previewLayout in sonos_widget_info.xml with a static XML layout. The Glance 1.2 providePreview() API was unverifiable on the 1.2.0-rc01 artifact (see decision E8). If Glance stable ships a confirmed providePreview(), migration is trivial.
- **Testing:** Glance widgets can only be tested by deploying to a device or emulator. There is no headless Glance rendering test framework.
- **Cleartext HTTP:** UPnP/SOAP uses HTTP over the local network. network_security_config.xml restricts cleartext to local IP ranges.
- **Foreground service types (Android 14+ hard crash):** All foreground services MUST declare android:foregroundServiceType in the <service> tag AND request the matching typed permission. Without both, startForeground() throws a SecurityException immediately.
- **Volume slider fragility:** Glance's native slider can exhibit unreliable drag behavior. Phase 1 implemented the [–] % [+] button fallback (5% steps, haptic per tap). Do not revisit the slider unless specifically requested.
- **Local Network Permission (Android 16):** All LAN device access requires the NEARBY_WIFI_DEVICES runtime permission. If not requested before discovery, all local Sonos calls will silently fail with zero results.

20.5 Verification Between Tasks

- **Compilation:** ./gradlew assembleDebug passes with zero errors and zero warnings. Requires JAVA_HOME and ANDROID_HOME to be set.

- **Installation:** The APK installs on the Pixel 9 (Android 16) or emulator with API 36 without crashes.
- **Visual check:** The widget renders correctly at the expected size(s) for the current phase.
- **Functional smoke test:** The specific acceptance criteria for the completed task are verified manually.

20.6 Reference Material for Agents

- **Jetpack Glance 1.2:** developer.android.com/jetpack/compose/glance. Pay special attention to the “Limitations” section.
- **Android 16 behavior changes:** developer.android.com/about/versions/16/behavior-changes-16. Critical for Local Network Protections.
- **Android 16 features and APIs:** developer.android.com/about/versions/16/features.
- **Sonos UPnP:** The Sonos device description XML at http://<speaker-ip>:1400/xml/device_description.xml provides the service list and SOAP action definitions.
- **Sonos Cloud API:** developer.sonos.com for REST endpoint documentation, OAuth flow, and rate limit details.
- **SSDP Protocol:** UPnP Device Architecture 2.0, Section 1 (Discovery). The M-SEARCH multicast address is 239.255.255.250:1900.
- **Android Palette API:** developer.android.com/reference/androidx/palette/graphics/Palette.